Trabajo Practico Integrador

Objetivo

- Desarrollar un sistema simple aplicando lo aprendido en la asignatura utilizando el Visual Studio y tecnologías .Net.
- Aplicar conocimientos incorporados en asignaturas que se relacionan.

Sistema

El dominio del sistema a construir queda a elección del grupo, tienen libertad de elegir lo que les resulte mejor, nuestra recomendación es que traten de construir un producto que pueda tener utilidad en la vida real.

Dicho esto, la cátedra pondrá a disposición de los alumnos un modelo de datos pensado para un sistema de gestión académica. Este modelo es solo de referencia, aún si deciden usarlo no es necesario seguirlo tal cual está planteado, pueden hacer todos los cambios que consideren necesarios siempre y cuando les sirva para cumplir con los requerimientos planteados.

Requerimientos Funcionales

- Tiene que incluir autenticación, esto es, un login que controle que solo puedan ingresar al sistema usuarios existentes.
- Tiene que contener al menos dos tipos de usuario y cada uno de esos tipos de usuario tiene que tener acceso a un grupo de funcionalidades y a otras no, o sea, tiene que haber autorización por tipo de usuario. Por ejemplo, si usan el ejemplo del sistema de gestión académica seguramente tengan usuarios que son alumnos y usuarios que son profesores.
- Tiene que tener como mínimo 6 CRUD/ABMCs.
- Uno de los CRUD/ABMCs debe incluir una búsqueda con filtros y otro que aplique el patrón relacional Maestro/Detalle (Ejemplo FacturaCabecera/FacturaDetalle).
- Tiene que tener como mínimo 2 Reportes y uno de los reportes tiene que incluir un gráfico.

Requerimientos Técnicos

- Tiene que funcionar para Desktop (o Mobile) y para Web.
- Para el acceso a datos es necesario usar ADO.NET al menos una vez. Para el resto puede usarse Entity Framework.
- Tiene que tener validaciones a nivel UI y a nivel Modelo de Dominio o Lógica de Negocios.

Arquitectura

El sistema, como mínimo, tiene que dividir las responsabilidades en los siguientes componentes:

- UI Desktop
- UI Web
- Servicios
- Modelo de Domínio / Lógica de Negócios
- Acceso a Datos

Cada uno de estos componentes va a estar representado por uno o más Proyectos de Visual Studio.

Draft

Entregas y Alcances

Alcances requeridos para cada entrega del Trabajo Práctico Integrador (TPI).

Cada grupo que opte por un dominio propio debe consensuar previamente su propuesta con el equipo docente de su comisión.

IMPORTANTE: las **fechas límite de cada entrega se encuentran publicadas en la Planificacion de Catedra**, presentada junto al cronograma para el ciclo actual.

Entrega 1: Servicios + Persistencia en Memoria.

- Implementar una solución tomando de referencia la arquitectura propuesta (en el Ejemplo 1), incluyendo proyectos que representen:
    - Modelo de Dominio / Lógica de Negocios
    - Capa de Servicios
- Desarrollar el **CRUD** de al menos **dos (2) entidades**, utilizando almacenamiento en memoria (que será reemplazado posteriormente por persistencia en base de datos).
- Implementar los servicios y exponerlos mediante Swagger, el cual será utilizado como interfaz (tipo consola) para realizar las pruebas.

Entrega 2: Servicios + Entity Framework + Interfaz Escritorio (Windows Forms)

- Hacer los cambios y agregados necesarios en la arquitectura tomando de referencia la propuesta por la cátedra (en el Ejemplo 2).
- Incorporar persistencia de datos utilizando **Microsoft SQL Server y Entity Framework**. La base de datos debe autogenerarse en caso de no existir.
- Desarrollar una **interfaz de escritorio** (**Windows Forms**), que permita operar los CRUD implementados en la entrega previa. Incluir:
    - Pantalla principal (con menú de navegación).
    - Formularios para cada entidad.
    - Funcionalidad de login/logout.

Entrega 3: Servicios + Entity Framework + Escritorio + Web (Blazor)

- Mantener la arquitectura propuesta.
- Replicar en una **interfaz Web (Blazor)** las funcionalidades implementadas previamente.
- Ampliar la solución incorporando (sólo en la aplicación de escritorio) un CRUD que aplique el patrón relacional **Maestro/Detalle** (ejemplo: Factura / Detalle de Factura).
- Implementar solo en Escritorio, CRUD adicional de entidades que apliquen patrón relacional Maestro/Detalle (Ej.: Factura/Detalle).
- Incorporar **seguridad mediante el uso de tokens**.

Entrega Final: Implementación completa del sistema

La entrega final deberá cumplir con el 100% de los requerimientos funcionales y técnicos, asegurando una solución estable y libre de errores. Además, deberá:

- Respeto de la arquitectura definida en la consigna.
- Implementación completa de todos los CRUDs en cada interfaz (Escritorio y Web).
- Desarrollo de dos (2) reportes en cada interfaz, donde:
    - Al menos uno debe incluir un gráfico.
    - Puede utilizarse cualquier motor de reportes.

Draft

- Implementación de seguridad completa:
    - Autenticación basada en tokens
    - Autorización por roles/perfiles de usuario
- Inclusión de datos de prueba representativos, suficientes para validar toda la funcionalidad del sistema.

Consideración sobre modalidades de aprobación

IMPORTANTE: Si bien **no existen diferencias en los alcances** entre las modalidades de Regularidad y Aprobación Directa (AD), sí existen diferencias en las **fechas de entrega** explicitadas en el Cronograma presentado en la Planificación de Cátedra.

Entregables

La entrega final deberá incluir:

1. Un **documento** (formato Word o equivalente) que contemple:
    a. **Carátula** con datos Institución, Asignatura, Comisión, Docentes, Grupo e Integrantes
    b. **Descripción general del sistema**, incluyendo:
        i. Requerimientos funcionales y no funcionales
    c. **Capturas de pantalla** de:
        i. Interfaces de escritorio
        ii. Interfaces web
        iii. Reportes
    d. **Modelo de datos** (Diagrama Tablas y Relaciones Base de Datos)
    e. **Modelo de clases**
    f. **Arquitectura implementada** (con breve descripción)
    g. **Tecnologías utilizadas** (incluir desde VS a librerías específicas/motor de reportes)
    h. **Enlace al repositorio de código fuente,** donde se encuentre publicada la solución desarrollada.
        i. El repositorio debe incluir la solución completa y **datos de prueba verosímiles,** suficientes para permitir la correcta ejecución y validación del sistema**.**
2. El **código fuente** del sistema, entregado en formato comprimido (por ejemplo, .zip), asegurando que sea posible su correcta apertura y ejecución. Recuerde no incluir las carpetas .suo, bin y obj.

Draft